WEEK 03

# Working with GPIO and CMSIS as HAL

Inside the chip at last. Clock gating, multiplexers, and how a line of C becomes a voltage on a pin

Textbook: Dean, *Embedded Systems Fundamentals*, 2nd ed., Chapter 2, printed pages 32–56.
Lab manual: Lab 03 Sections 3.1–3.11 (GPIO architecture, HAL and BSP functions, ODR writes).

## 1 What we are actually after this week

Last week we stood outside the chip with a multimeter. This week we go in, and the question we are answering is embarrassingly basic and surprisingly deep: **when you write a line of C, how does a voltage appear on a pin?**

The answer is a chain, and we are going to walk it link by link:

1. **Control registers.** Peripherals are configured by writing to special memory addresses. What those look like.
2. **CMSIS.** Nobody memorises addresses. The layer that gives them names.
3. **Bit fields, and how to change one without wrecking its neighbours.** This is where most beginner bugs live.
4. **Clock gating.** The step that, when forgotten, makes a perfectly correct program do absolutely nothing.
5. **The pin multiplexer.** Why one physical pin can be a GPIO, or a UART line, or a timer output, and how you choose.
6. **The GPIO register set.** MODER, IDR, ODR, and the rest.
7. **BSRR**, the register that exists purely to make your life easier and faster, and which HAL secretly uses.
8. The complete switch-and-LED program, assembled.
9. Active-high versus active-low wiring, which flips your code's logic upside down.
10. The remaining pin options: pull resistors in software, push-pull versus open drain, drive speed, and configuration lock.

Items 3, 4, and 7 are the ones that will save you actual hours of debugging. Item 5 is the one that turns into an exam question. Items 9 and 10 get lighter treatment but they are not skippable, because item 9 is the difference between your LED working and your LED being inverted.

This is the longest week so far. Let's go.

## 2 Deep dive 1: Control registers are just memory

Recall that a peripheral is a hardware circuit sitting next to the CPU. But the CPU only knows how to do a few things: read from memory, write to memory, do arithmetic. So how does it talk to a circuit?

The trick is that it does not, quite. The peripheral's configuration is exposed as **control registers** that live at specific memory addresses. The CPU writes a number to an address, and the hardware at that address reconfigures itself. From the CPU's point of view it was an ordinary memory write. From the peripheral's point of view it was an instruction.

**Control register**

> A register used to configure the operation of hardware in the CPU or a peripheral, exposed at a fixed memory address. Reading it usually tells you the current configuration or status. Writing it changes behaviour. This scheme is called **memory-mapped I/O**, and it is why "everything is memory" is a useful lie to believe on an MCU.

The reference manual documents these in tables that look like this, taken from the RCC (Reset and Clock Controller) peripheral:

| Absolute address | Register | Width | Reset value |
|---|---|---|---|
| 4002_1000 | Clock control register (RCC_CR) | 32 | 0000_XX83h |
| 4002_1004 | Clock configuration register (RCC_CFGR) | 32 | 0000_0000h |
| 4002_1014 | AHB peripheral clock enable (RCC_AHBENR) | 32 | 0000_0014h |
| 4002_101C | APB peripheral clock enable 1 (RCC_APB1ENR) | 32 | 0000_0000h |

Three things to notice, because you will be reading tables like this all semester. Addresses are in **hexadecimal**, base sixteen, where each digit runs 0–9 then A–F for ten through fifteen. The **reset value** tells you what the register contains after power-on, and an `X` means those bits are undefined. And the naming convention is rigid: the peripheral name, an underscore, then the register name. `RCC_AHBENR` is the `AHBENR` register of the `RCC` peripheral.

## 3 Deep dive 2: CMSIS, so you never type an address

Now, you could write to `0x40021014` directly with a cast to a volatile pointer. People did this for years. It is miserable, unreadable, and one typo produces a bug that takes an afternoon.

So instead:

**CMSIS**

> The **Cortex Microcontroller Software Interface Standard**, a hardware abstraction layer for Cortex-M processors. Its **CMSIS-CORE** component provides a C interface to the processor core and peripherals: macros, functions, and C data structures whose layout matches the registers exactly.

The mechanism is elegant once you see it. For each peripheral, there is a `struct` whose members sit at exactly the right offsets, and a pointer to that struct located at the peripheral's base address. So:

Listing 1: The GPIO register structure. This is what makes `GPIOA->MODER` work.

```
typedef struct {
    __IO uint32_t MODER;    /* mode register,           offset 0x00 */
    __IO uint32_t OTYPER;   /* output type register,    offset 0x04 */
    __IO uint32_t OSPEEDR;  /* output speed register,   offset 0x08 */
    __IO uint32_t PUPDR;    /* pull-up/pull-down,       offset 0x0C */
    __IO uint32_t IDR;      /* input data register,     offset 0x10 */
    __IO uint32_t ODR;      /* output data register,    offset 0x14 */
    __IO uint32_t BSRR;     /* bit set/reset register,  offset 0x18 */
```

```
    __IO uint32_t LCKR;      /* configuration lock,        offset 0x1C */
    __IO uint32_t AFR[2];    /* alternate function L/H,    offset 0x20-0x24 */
    __IO uint32_t BRR;       /* bit reset register,        offset 0x28 */
} GPIO_TypeDef;
```

`GPIOA` is defined as a `GPIO_TypeDef *` pointing at the port's base address, which is why you write `GPIOA->MODER` with an arrow rather than a dot. The compiler turns that into "base address plus offset 0x00", which is precisely the address the reference manual listed. You get readable names and the compiler does the arithmetic.

That `__IO` qualifier expands to `volatile`, and it matters enormously. It tells the compiler that this memory can change without the program touching it, so the compiler must not cache the value in a CPU register or optimise away a read that looks redundant. Strip the `volatile` off a status register and your polling loop will hang forever, reading a stale copy.

**Book board vs your board**

> The header file you include names the exact chip.
>
> Book: `#include <stm32f091xc.h>`
> Yours: `#include "stm32f3xx.h"` or `#include "stm32f303xc.h"`
>
> This is the **device peripheral access layer (DPAL)**. Above it, ST layers two more things: the **LL (low-layer) drivers**, thin inline functions close to the registers, and the **STM32Cube HAL**, thicker portable functions. Your lab manual uses HAL almost exclusively. The book uses DPAL almost exclusively. We will do both, because understanding the registers is what lets you debug HAL when it misbehaves.

# 4 Deep dive 3: Bit fields, and the read-modify-write discipline

Here is the problem that generates more beginner bugs than anything else in this course.

A 32-bit control register does not hold one setting. It holds many, packed into **fields** of one or more bits each. In `RCC_AHBENR`, bit 0 is `DMAEN`, bit 1 is `DMA2EN`, bit 2 is `SRAMEN`, bit 4 is `FLITFEN`, bit 6 is `CRCEN`, and bits 17 through 22 enable the clocks for GPIOA through GPIOF. Various other bits are **reserved**, which means the manual is telling you to leave them alone.

So when you want to enable one thing, you must change one bit and leave thirty-one others exactly as they were.

## 4.1 The three symbols CMSIS gives you per field

Vendors define names following the pattern `<peripheral>_<register>_<field>`, and for each field you get three symbols:

- `_Pos` is the **position** of the field's least significant bit. `RCC_AHBENR_FLITFEN_Pos` is 4.
- `_Msk` is a **mask** with ones in the field's positions and zeros elsewhere. `RCC_AHBENR_FLITFEN_Msk` is `0x00000010`.
- The bare name with **no suffix** equals the `_Msk` value, provided purely so the common case reads nicely.

Listing 2: How the header defines them. Note the differing widths.

```
#define RCC_AHBENR_FLITFEN_Pos    (4U)
#define RCC_AHBENR_FLITFEN_Msk    (0x1U << RCC_AHBENR_FLITFEN_Pos)
#define RCC_AHBENR_FLITFEN        RCC_AHBENR_FLITFEN_Msk

#define GPIO_MODER_MODER1_Pos     (2U)
#define GPIO_MODER_MODER1_Msk     (0x3U << GPIO_MODER_MODER1_Pos)
```

```
#define GPIO_MODER_MODER1         GPIO_MODER_MODER1_Msk
```

Look closely at the difference. The RCC mask shifts `0x1`, one bit, because clock-enable is a single-bit field. The MODER mask shifts `0x3`, which is `0b11`, two bits, because **each pin's mode takes two bits**. That two-bit width is going to matter in a moment.

**Beware the trap**

> The header sometimes disagrees with the reference manual on field names. The reference manual for the F0 calls the GPIO clock-enable bits `IOPAEN` through `IOPFEN`, but the header defines them as `RCC_AHBENR_GPIOAEN` and so on. The bit positions are identical, only the names differ.
>
> The reliable procedure when in doubt: open the header, search for the `_Pos` symbol whose value matches the bit position the manual gives you, and use whatever name that symbol has. The header is what the compiler sees, so the header wins.

## 4.2 Setting a single bit

For a one-bit field, OR it in:

```
RCC->AHBENR |= RCC_AHBENR_GPIOAEN;     // set bit 17, leave the rest alone
```

The `|=` is what preserves the other bits. And if you are building masks by hand, the idiom is:

```
#define MASK(x)  (1UL << (x))

n = MASK(17) | MASK(18);      // enable GPIOA and GPIOB clocks
```

**Why `1UL` and not just `1`**

> The registers are 32 bits, so the compiler must produce a 32-bit unsigned value. A bare `1` is a plain `int`, and shifting it left by 31 is undefined behaviour on a signed type. The `UL` suffix forces *unsigned long*, which is 32 bits on Cortex-M, so the shift is well defined all the way to bit 31. Write `1UL`, always. It costs one keystroke and removes a class of bug that only appears on high bit numbers.

## 4.3 Setting a multi-bit field, which is the harder case

Now the two-bit MODER field. You cannot just OR, because if the field currently holds `11` and you OR in `01`, you get `11`. OR can only turn bits on. You must clear the field first, then set it. Three steps:

Listing 3: Read-modify-write, spelled out.

```
GPIOA->MODER &= ~GPIO_MODER_MODER5_Msk;          // 1. clear the field
GPIOA->MODER |= (1UL << GPIO_MODER_MODER5_Pos);  // 2. set the new value
```

Because this is tedious and easy to fumble, the book wraps it in a macro:

Listing 4: `MODIFY_FIELD`, from the book's `field_access.h`.

```
#define MODIFY_FIELD(reg, field, value)                    \
    ((reg) = ((reg) & ~(field ## _Msk))                    \
           | (((uint32_t)(value) << field ## _Pos) & field ## _Msk))
```

The `##` is the C preprocessor's token-paste operator, so passing `GPIO_MODER_MODER5` as `field` produces `GPIO_MODER_MODER5_Msk` and `GPIO_MODER_MODER5_Pos` automatically. Usage collapses to one clear line:

```
MODIFY_FIELD(GPIOA->MODER, GPIO_MODER_MODER5, 1);   // PA5 -> output
```

CMSIS also ships `_VAL2FLD` and `_FLD2VAL` for the same job. `_VAL2FLD` takes a value aligned at bit 0, shifts it up to the field's position, and masks off anything that overflowed. `_FLD2VAL` does the reverse: masks the field out of a register value and shifts it back down to bit 0 so you can compare it against a plain number.

**Beware the trap**

Notice the extra `& field_Msk` at the end of `MODIFY_FIELD`. That is not decoration. If you pass a value too large for the field, say 7 into a 2-bit field, the shifted value spills into the neighbouring field and silently reconfigures the pin next door. The mask truncates it instead. Do not write your own version of this macro without that final AND.

## 5 Deep dive 4: Clock gating, the step everybody forgets once

Right. Before any of the above works, there is a prerequisite, and it is the single most common reason a beginner's GPIO program does nothing at all.

Between the CPU and a pin sits a chain: the **RCC** supplies a clock to peripheral modules, the **GPIOx** module decides which peripheral owns each pin, and the peripheral itself exchanges data with the CPU. The RCC's job is deliberately stingy. It supplies a clock *only to modules that have been switched on*.

**Clock gating**

Disabling a circuit by blocking its clock signal, so its transistors stop switching and it stops consuming dynamic power. A gated peripheral is not merely idle, it is inert: its registers do not respond. On the STM32 this is controlled by the RCC's `AHBENR`, `APB1ENR`, and `APB2ENR` registers, with one enable bit per peripheral.

So the very first line of any GPIO setup is:

```
RCC->AHBENR |= RCC_AHBENR_GPIOAEN;     // give port A a clock
```

**Beware the trap**

Here is the failure mode, and it is nasty because it is silent. Write to `GPIOA->MODER` *before* enabling the GPIOA clock and the write does not fault, does not warn, and does not take effect. It vanishes. Then you configure everything else correctly, run the program, and the LED never lights.

Worse, if you then set a breakpoint and read `GPIOA->MODER` back, you may read zeros and conclude your macro is broken. It is not. The port was asleep.

Rule: **clock first, configure second, always.** If a peripheral appears completely dead, check its clock enable bit before you check anything else. This costs you thirty seconds and it is the correct first move roughly half the time.

**Book board vs your board**

The good news: GPIO clock enabling is essentially identical on both parts. Both put the GPIO ports on the AHB bus, both use `RCC->AHBENR`, and the bit positions match, with `GPIOAEN` at bit 17 running up through `GPIOFEN` at bit 22.

Your lab manual uses the HAL macro instead, which is what CubeMX generates:

```
__HAL_RCC_GPIOA_CLK_ENABLE();      // HAL way
RCC->AHBENR |= RCC_AHBENR_GPIOAEN;  // register way, identical effect
```

The manual is blunt about it and correct: enabling the port clock is **mandatory before**

**any GPIO operation**.

# 6 Deep dive 5: The pin multiplexer, or how one pin does six jobs

Now the second piece of preparation. A clock is not enough. The GPIO signals inside the silicon have to be *routed* out to a physical pin.

Ask yourself why this is even necessary. An STM32F303VCT6 has up to 87 usable I/O pins, and internally it has far more peripheral signals than that: several UARTs, several SPIs, several $I^2C$ buses, a dozen timers with multiple channels each, four ADCs. If every internal signal needed its own pin, the package would be enormous and expensive.

So instead there is a **multiplexer** on every pin.

**Multiplexer**

An electronic selector switch that routes one of $N$ input signals to an output. A pin multiplexer on an MCU is bidirectional, since a pin may be input or output. It lets one physical pin be connected to one of several possible peripheral modules, chosen in software, which shrinks package size, board size, and cost.

The STM32 gives each pin *two* multiplexers in series.

## 6.1 The first mux: MODER, choosing the broad category

The mux closest to the pin picks one of four categories, set by a two-bit field `MODERn` in `GPIOx_MODER`:

| `MODERn[1:0]` | Configuration | Book's symbol |
|---|---|---|
| `00` | Input | `ESF_GPIO_MODER_INPUT` |
| `01` | Output | `ESF_GPIO_MODER_OUTPUT` |
| `10` | Alternate function | `ESF_GPIO_MODER_ALT_FUNC` |
| `11` | Analog | `ESF_GPIO_MODER_ANALOG` |

Two bits per pin, sixteen pins, so `MODER` uses all 32 bits: `MODER0` at bits [1:0], `MODER1` at bits [3:2], and generally `MODERn` at bits $[2n{+}1 : 2n]$. That is where the factor of two in `GPIO_MODER_MODER1_Pos = 2` came from.

The book's `ESF_` prefix is its own invention, used to keep its symbols distinct from ST's. ST's HAL defines similar values named `GPIO_MODE_INPUT`, `GPIO_MODE_OUTPUT_PP` and so on, but those bundle in extra features which muddies the explanation, so the book rolls its own.

## 6.2 The second mux: AFR, choosing which alternate function

If, and only if, `MODERn` is `10` (alternate function), a second mux picks *which* alternate function, out of up to eight. That is controlled by a four-bit `AFSELn` field, and because sixteen pins times four bits is 64 bits, it takes two registers:

- `GPIOx_AFRL` covers pins 0 through 7
- `GPIOx_AFRH` covers pins 8 through 15

In CMSIS these appear as the array `AFR[2]`, so `AFR[0]` is AFRL and `AFR[1]` is AFRH.

What each `AFSEL` value means is **different for every pin**, and there is no pattern to memorise. The datasheet has an alternate-function table and you look it up. To give you a feel for it, three pins on the book's part:

| AFSEL | PA1 | PA2 | PA5 |
|---|---|---|---|
| 0000 | EVENTOUT | TIM15_CH1 | SPI1_SCK, I2S1_CK |
| 0001 | USART2_RTS | USART2_TX | CEC |
| 0010 | TIM2_CH2 | TIM2_CH3 | — |

**In the lab**

This is exactly the mechanism CubeMX is operating when you click a pin and pick `USART2_TX` from a dropdown. It sets `MODER` to alternate function and writes the right `AFSEL` value, having looked up the table for you.

Which is genuinely useful, and also why students who only ever use CubeMX are helpless when a pin does not work. If your UART is silent in Lab 02, or your PWM pin is dead in Lab 05, the diagnosis is almost always one of three things: the port clock is off, `MODER` is not set to alternate function, or `AFSEL` holds the wrong number for that specific pin. Check them in that order.

## 7 Deep dive 6: The GPIO register set

With the plumbing understood, here is the whole port. All ten registers, all 32 bits wide, at fixed offsets from the port base:

| Offset | Register | Purpose |
|---|---|---|
| 0x00 | MODER | Mode: input, output, alternate, analog. 2 bits/pin |
| 0x04 | OTYPER | Output type: push-pull or open drain. 1 bit/pin |
| 0x08 | OSPEEDR | Output slew rate. 2 bits/pin |
| 0x0C | PUPDR | Internal pull-up / pull-down. 2 bits/pin |
| 0x10 | IDR | **Input data.** Read-only. Bit $n$ is pin $n$'s level |
| 0x14 | ODR | **Output data.** Bit $n$ drives pin $n$ |
| 0x18 | BSRR | Bit set/reset. Atomic writes. See Section 8 |
| 0x1C | LCKR | Configuration lock |
| 0x20-0x24 | AFRL, AFRH | Alternate function select. 4 bits/pin |
| 0x28 | BRR | Bit reset only |

The two you use constantly are `IDR` and `ODR`. Reading a pin means testing a bit of `IDR`. Writing a pin means setting a bit of `ODR`.

Underneath, a single port bit is genuinely simple hardware: a *data in* register that the I/O clock samples continuously, perhaps 48 million times a second, whose contents get gated onto the data bus when the CPU reads `IDR`; and a *data out* register that latches whatever the CPU writes to `ODR` and presents it to the pin driver. Sixteen copies of that, side by side, make a port.

Note the asymmetry in width. Each port has at most 16 pins, but every register is 32 bits, because MCU designers like port registers to match the processor's native word size. So the upper half of `IDR` and `ODR` reads as zeros and is not useful.

**Beware the trap**

Not every port has all sixteen pins brought out. The die supports up to 16 per port, but the package does not have enough legs. The datasheet's pinout section tells you which port bits actually exist on your package. On a 100-pin STM32F303VCT6 most of ports A through E are available and port F is sparse. Writing to a `MODER` field for a pin that does not physically exist is harmless and also achieves nothing, which makes for a confusing afternoon.

### 7.1 The naive program

Putting `IDR` and `ODR` together, here is a first attempt at "light the LED while the button is held":

Listing 5: Read-modify-write on ODR. Works, but see the problem below.

```
#define LD2_POS  (5)
#define B1_POS   (13)
#define MASK(x)  (1UL << (x))

GPIOA->ODR &= ~MASK(LD2_POS);          // start with the LED off

while (1) {
    if (GPIOC->IDR & MASK(B1_POS)) {
        GPIOA->ODR &= ~MASK(LD2_POS);  // 1 = not pressed -> LED off
    } else {
        GPIOA->ODR |= MASK(LD2_POS);   // 0 = pressed     -> LED on
    }
}
```

This is correct, and it is also the last time we will write it this way, because of the register in the next section.

## 8 Deep dive 7: BSRR, and why atomicity is worth a whole register

Look hard at `GPIOA->ODR |= MASK(LD2_POS)`. It looks like one operation. It is three: load `ODR` into a CPU register, OR in the bit, store it back. Two bus accesses with a gap in the middle.

Now, what if something happens in that gap?

Suppose an interrupt fires between the load and the store, and its handler turns on a different LED on the same port by doing its own read-modify-write. The handler reads `ODR`, sets its bit, writes back. Then control returns to your code, which stores *the value it read before the interrupt*, silently erasing the handler's change. The other LED flickers on and off for no reason anybody can see, once every few thousand iterations.

This is a race condition, and it is exactly the kind of bug that makes people distrust hardware. The GPIO peripheral solves it with dedicated hardware.

**BSRR, the bit set/reset register**

`GPIOx_BSRR` lets you change specific output bits without touching the others, in a **single atomic write**. Its 32 bits split into two halves:

· Writing 1 to bit $i$ of the **lower** half (`BS`$i$, bits 0–15) **sets** `ODR` bit $i$.
· Writing 1 to bit $i$ of the **upper** half (`BR`$i$, bits 16–31) **resets** `ODR` bit $i$.
· Writing 0 anywhere has **no effect**.

That last rule is the whole trick. Because zeros do nothing, you never have to preserve the existing contents, so there is nothing to read first.

Two examples. To set the low byte of port A to all ones, write `0x000000FF` to `GPIOA->BSRR`.

To clear the low byte to all zeros, write `0x00FF0000`. In both cases a plain assignment, no OR, no read.

The program becomes:

Listing 6: Same behaviour, atomic, and one bus access per change.

```
GPIOA->BSRR = GPIO_BSRR_BR_5;                // LED off

while (1) {
    if (GPIOC->IDR & GPIO_IDR_13) {
        GPIOA->BSRR = GPIO_BSRR_BR_5;        // not pressed -> off
    } else {
        GPIOA->BSRR = GPIO_BSRR_BS_5;        // pressed     -> on
    }
}
```

Notice the `=` rather than `|=`. That is not sloppiness, it is the point. No read, no modify, just a write.

**In the lab**

Here is the payoff that ties this to your labs. Open the STM32F3 HAL source for `HAL_GPIO_WritePin` and you will find, in essence:

```
if (PinState != GPIO_PIN_RESET)  GPIOx->BSRR = GPIO_Pin;
else                             GPIOx->BRR  = GPIO_Pin;
```

It is a wrapper around `BSRR` and `BRR`. So every time you call `HAL_GPIO_WritePin` in Lab 03, you are using this register whether you know it or not.

Which means the manual's Section 3.10 optimisation tip, `GPIOA->ODR = (GPIOA->ODR & ~0x7F) | pattern;`, is actually a *step backwards* in one respect: it reintroduces the read-modify-write that HAL was avoiding. For a seven-segment display driven from `main` with nothing else touching port A, that is perfectly safe. Once you add the timer interrupt from Lab 04, it stops being safe. The atomic equivalent writes both halves of BSRR at once:

```
GPIOA->BSRR = ((uint32_t)(~pattern & 0x7F) << 16) | (pattern & 0x7F);
```

Set the bits you want high in the low half, reset the rest in the high half, one write, no race.

## 9 Deep dive 8: The complete program

Now assemble everything. Clock, mode, then loop.

Listing 7: The full switch-and-LED program, register level. Book Listing 2.8, lightly annotated.

```
#include <stm32f091xc.h>          /* yours: "stm32f3xx.h" */

#define LD2_OFF_MSK   (GPIO_BSRR_BR_5)
#define LD2_ON_MSK    (GPIO_BSRR_BS_5)
#define B1_IN_MSK     (GPIO_IDR_13)

void Basic_Light_Switching_Example(void) {
    /* 1. Clock the ports. Nothing below works without this. */
    RCC->AHBENR |= RCC_AHBENR_GPIOAEN;     /* for LD2 on PA5  */
    RCC->AHBENR |= RCC_AHBENR_GPIOCEN;     /* for B1  on PC13 */

    /* 2. Route the pins: PA5 output, PC13 input. */
    MODIFY_FIELD(GPIOA->MODER, GPIO_MODER_MODER5,  ESF_GPIO_MODER_OUTPUT);
    MODIFY_FIELD(GPIOC->MODER, GPIO_MODER_MODER13, ESF_GPIO_MODER_INPUT);

```

```
    /* 3. Known starting state. */
    GPIOA->BSRR = LD2_OFF_MSK;

    /* 4. Poll forever. */
    while (1) {
        if (GPIOC->IDR & B1_IN_MSK) {
            GPIOA->BSRR = LD2_OFF_MSK;     /* 1 = released */
        } else {
            GPIOA->BSRR = LD2_ON_MSK;      /* 0 = pressed  */
        }
    }
}

int main(void) {
    Basic_Light_Switching_Example();
}
```

Four phases, and they are the same four phases for every peripheral you will meet this semester: **clock it, route it, initialise it to a known state, then use it**. Timers in week 12, ADC in week 11, SPI in week 15. Same shape every time.

**Beware the trap**

The book's own Listing 2.8 contains a bug worth learning from. It writes

```
RCC->AHBENR = RCC_AHBENR_GPIOAEN;     /* note: = not |= */
RCC->AHBENR = RCC_AHBENR_GPIOCEN;     /* this wipes GPIOA! */
```

Plain assignment, twice. The second line clears the GPIOA enable bit that the first line set, along with every other clock enable in that register, including `SRAMEN` and `FLITFEN` which are set at reset for a reason.

It happens to survive on that board because the port A clock gets re-enabled elsewhere, but the pattern is wrong. **Clock enable registers are always `|=`, never `=`.** If you are reading the book's listings alongside these notes, this is the one line to mentally correct.

**Book board vs your board**

The same program on your board, in the HAL style your labs use:

```
GPIO_InitTypeDef g = {0};

__HAL_RCC_GPIOE_CLK_ENABLE();           /* LD3..LD10 live on port E */
__HAL_RCC_GPIOA_CLK_ENABLE();           /* user button on PA0       */

g.Pin   = GPIO_PIN_9;                   /* one of the user LEDs     */
g.Mode  = GPIO_MODE_OUTPUT_PP;          /* push-pull output         */
g.Pull  = GPIO_NOPULL;
g.Speed = GPIO_SPEED_FREQ_LOW;
HAL_GPIO_Init(GPIOE, &g);

g.Pin  = GPIO_PIN_0;
g.Mode = GPIO_MODE_INPUT;
g.Pull = GPIO_PULLDOWN;                 /* your button is active HIGH */
HAL_GPIO_Init(GPIOA, &g);

while (1) {
    HAL_GPIO_WritePin(GPIOE, GPIO_PIN_9,
        HAL_GPIO_ReadPin(GPIOA, GPIO_PIN_0) ? GPIO_PIN_SET :
            GPIO_PIN_RESET);
}
```

Note the polarity flip we flagged in Week 02. The book's button reads 0 when pressed, so

its logic is inverted. Yours reads 1 when pressed, so it is not. Same code shape, opposite test.

Also note `HAL_GPIO_Init` does in one call what `MODIFY_FIELD` did to `MODER`, plus `PUPDR`, plus `OSPEEDR`, plus `OTYPER`, plus `AFR` if you asked for an alternate function. Convenient, and considerably more instructions.

## 10 Deep dive 9: Active low, and why your LED might be inverted

A short but important section, because it silently inverts programs.

In the circuit from Week 02, the LED's *anode* went to the MCU pin through a resistor and its cathode went to ground. Output high lights it. That is **active high**.

The book's RGB LED example wires it the other way: the LED *cathodes* connect through resistors to pins PA5, PA6, PA7, and the anodes go to the supply. Now output *low* lights the LED and output high turns it off. That is **active low**.

Why would anyone do that? Because many MCU output drivers can **sink** more current than they can **source**. The transistor pulling the pin down to ground is often beefier than the one pulling it up to $V_{DD}$. So wiring the LED to be lit by a low output gets you a brighter LED within the same pin ratings.

Listing 8: Active-low definitions. Read them carefully, the names look backwards on purpose.

```
/* 0 out = LED on, 1 out = LED off */
#define LED_R_OFF_MSK   (GPIO_BSRR_BS_5)    /* SET the pin to turn OFF */
#define LED_R_ON_MSK    (GPIO_BSRR_BR_5)    /* RESET the pin to turn ON */
```

And the parallel version of the flasher, which shows the inversion arithmetically:

Listing 9: Writing three LED bits at once. Note the `~num`.

```
void RGB_Flasher_Parallel(void) {
    unsigned int num = 0;
    while (1) {
        num++;
        GPIOA->ODR &= ~((0x07) << 5);          /* clear the 3-bit field    */
        GPIOA->ODR |= (~num & 0x07) << 5;      /* complement: 1 -> 0 out   */
        Delay(400);
    }
}
```

That `~num` is the entire active-low correction. A 1 bit in `num` must produce a 0 on the output to light the LED, so you invert before writing.

**In the lab**

This is directly your Lab 03 problem. A **common anode** seven-segment display is active low: the shared anode sits at 3 V and you pull a segment pin low to light that segment. That is why the manual's lookup table has digit 8, all segments lit, as `0x00`.

So if you wire a common anode display and use a common cathode lookup table, you get a perfect photographic negative of every digit. If your display shows the inverse of what you expect, you do not have a code bug. You have a table that does not match your hardware, and the fix is one `~` or one different table.

**Beware the trap**

Your STM32F3 Discovery's own user LEDs are **active high**: they are wired anode to the pin, so `GPIO_PIN_SET` lights them, and `BSP_LED_On` does the obvious thing. The book's Nucleo LD2 is also active high, but its RGB example is active low, and a common-anode

> display is active low.
>
> So within one lab you may be juggling both conventions on the same board. Write the polarity down in a comment next to each pin definition. Not in your head.

# 11 Deep dive 10: The remaining pin options

Four more configuration registers, covered lightly, because you will meet each of them properly later.

## 11.1 PUPDR: pull-ups and pull-downs in software

Recall from Week 02 that a floating input reads garbage and needs a pull resistor. You do not always have to fit one, because every pin has weak internal pull-up and pull-down resistors selectable through `GPIOx_PUPDR`, two bits per pin: `00` none, `01` pull-up, `10` pull-down. In HAL that is the `Pull` member: `GPIO_NOPULL`, `GPIO_PULLUP`, `GPIO_PULLDOWN`.

They are weak, tens of kΩ, which is right for a button and wrong for anything that needs to hold a bus firmly.

## 11.2 OTYPER: push-pull versus open drain

One bit per pin. **Push-pull** is the normal case: the driver actively pulls the pin high and actively pulls it low, two transistors. **Open drain** removes the upper transistor, so the pin can only pull low or let go, floating when idle.

That sounds strictly worse, and for an LED it is. But it is exactly what a shared bus needs, because several open-drain devices can share one wire with a single pull-up resistor: any device can pull the line low, and the line only goes high when all of them let go. This is why $I^2C$ requires open drain, and it is the reason week 16 exists. Park the idea.

## 11.3 OSPEEDR: output slew rate

Two bits per pin, controlling how *fast* the driver transitions between levels. Fast edges are needed for high-frequency signals like an SPI clock, and they cost you: sharper edges radiate more electromagnetic interference and cause more ringing on a badly laid-out board. Default to low speed and raise it only when a peripheral needs it.

## 11.4 LCKR: configuration lock

Write a magic sequence to `GPIOx_LCKR` and the configuration of the selected pins is frozen until the next reset. Nothing, not even buggy code, can reconfigure them. Used in safety-critical designs where a pin controlling something dangerous must not be accidentally turned into a UART. You will not need it this semester, but recall the reliability attribute from Week 01: this is what that attribute looks like in a register.

And that is a wrap on Chapter 2. You now know, end to end, how a line of C becomes a voltage.

# 12 Tying it back

We opened by asking how a line of C becomes a voltage on a pin. Here is the whole chain in one breath.

The peripheral's behaviour is exposed as control registers at fixed memory addresses, so a write to memory is a command to hardware. CMSIS wraps those addresses in structs and their

bit fields in `_Pos` and `_Msk` symbols, so you write `GPIOA->MODER` instead of a hex constant, and `volatile` keeps the compiler from optimising the whole thing away. Because a register packs many unrelated settings, you change one field with read-modify-write, or with `MODIFY_FIELD` so the masking is done for you. But none of it takes effect until the RCC gates a clock through to that port, which is the failure everyone hits once. With the clock running, `MODER` picks whether the pin is input, output, alternate, or analog, and if alternate, `AFR` picks which of eight functions, which is what CubeMX is doing behind its dropdown. Then you read levels from `IDR` and drive them from `ODR`, or better, from `BSRR`, whose ignore-the-zeros design makes a bit change atomic and thereby immune to a race with an interrupt handler, which is exactly what `HAL_GPIO_WritePin` uses internally. And whether a 1 in your code means light or dark depends on which end of the LED faces the pin.

Next week the shape of the course changes. We stop asking how to control one pin and start asking how to do several unrelated things at once with a single CPU, which is where that busy-wait `Delay` function starts looking like the liability it is.

# Cheat sheet: Week 03

**The four phases, for every peripheral all semester**
**1. Clock it → 2. Route it → 3. Initialise to a known state → 4. Use it**
Skip phase 1 and phases 2–4 silently do nothing. No fault, no warning.

**CMSIS bit field symbols**
`<PERIPH>_<REG>_<FIELD>_Pos` = position of the field's LSB.
`<PERIPH>_<REG>_<FIELD>_Msk` = ones in the field, zeros elsewhere.
`<PERIPH>_<REG>_<FIELD>` = same as `_Msk`, for readability.
Header names can differ from the reference manual (`GPIOAEN` vs `IOPAEN`). **The header wins.**
`__IO` = `volatile`. Never remove it. `1UL` not `1`, so shifts to bit 31 are defined.

**Changing bits**
Single bit, set: `REG |= FIELD;` Single bit, clear: `REG &= ~FIELD;`
Multi-bit field, must read-modify-write:

```
REG &= ~FIELD_Msk; REG |= (value << FIELD_Pos);
```

Or `MODIFY_FIELD(reg, field, value)`, which masks the value so it cannot spill into the next field.
**Clock enable registers are always `|=`, never `=`.** Plain `=` wipes every other enable bit.

**GPIO registers, offsets from port base**
`0x00 MODER` 2b/pin `0x04 OTYPER` 1b/pin `0x08 OSPEEDR` 2b/pin `0x0C PUPDR` 2b/pin
`0x10 IDR` read pins `0x14 ODR` drive pins `0x18 BSRR` atomic `0x1C LCKR` freeze
`0x20/0x24 AFRL/AFRH` 4b/pin, as `AFR[0]/AFR[1]` `0x28 BRR` reset only
`MODERn` sits at bits $[2n{+}1 : 2n]$. `00` input, `01` output, `10` alternate function, `11` analog.
Registers are 32 bits but ports have at most 16 pins, so the upper half of `IDR/ODR` is unused.

**BSRR, and why it exists**
Lower half (bits 0–15), write 1 ⇒ **sets** that `ODR` bit.
Upper half (bits 16–31), write 1 ⇒ **resets** that `ODR` bit.
Writing 0 anywhere does nothing, so no read is needed, so the change is **atomic**.
Use `=` not `|=`. Example: `GPIOA->BSRR = 0x000000FF;` sets PA0–PA7. `0x00FF0000` clears them.
`HAL_GPIO_WritePin` is a wrapper around `BSRR/BRR`. An `ODR` read-modify-write can race with an ISR; `BSRR` cannot.

**Two multiplexers per pin**
First mux: `MODER` picks input / output / alternate / analog.
Second mux: `AFSEL` in `AFRL` or `AFRH` picks which of up to 8 alternate functions. **The meaning of each AFSEL value differs per pin** and lives in the datasheet's alternate function table.
Dead UART or dead PWM pin? Check in order: port clock, `MODER` = alternate, correct `AFSEL`.

**Active high vs active low**

Anode to pin, cathode to ground ⇒ **active high**, output 1 lights it.
Cathode to pin, anode to supply ⇒ **active low**, output 0 lights it. Used because pins usually *sink* more current than they *source*.
**Common anode seven-segment is active low**, which is why the manual's digit 8 is `0x00`.
Wrong table ⇒ every digit displays inverted.
Your board's user LEDs are active high. Write the polarity in a comment, not in your head.

**The other four registers, in one line each**

`PUPDR`: internal pull-up / pull-down, weak (tens of kΩ), replaces an external resistor for buttons.
`OTYPER`: push-pull (drives both ways) or open drain (pulls low or floats). **I²C needs open drain**, week 16.
`OSPEEDR`: slew rate. Fast edges for SPI clocks, more EMI. Default low.
`LCKR`: magic-sequence write freezes pin configuration until reset. Safety-critical use.

**Likely exam prompts from this chapter**

· Write register-level C to configure a named pin as an output and drive it high. *Near certain.*
· Why must the peripheral clock be enabled first, and what happens if it is not?
· Explain read-modify-write and why `|=` alone cannot set a 2-bit field to `01`.
· What problem does `BSRR` solve that `ODR` does not? Answer with atomicity and the ISR race.
· Given a pin number $n$, state which bits of `MODER` configure it. Answer: $[2n{+}1 : 2n]$.
· Why does an MCU multiplex peripherals onto shared pins?